COMP101
Introduction to Programming
2025-26

Lecture-18
Modularisation
(Python functions)

Modularisation – Python functions

- Modularisation is the process of sectioning code into functions

- Functions are smaller sections of code that perform specific tasks that can be ‘called’ and used repeatedly

- Modularisation uses several terms (basically all do the same thing)
    - ‘functions’, ‘procedures’
    - ‘routines’, ‘subroutines’,
    - ‘modules’, ‘methods’

- **Python uses ‘functions’**
    - **‘function definition’, ‘user-defined function’**

Modularisation – Python functions

- Functions and procedures are the most common terms
  - Functions perform a specific task and return a value
  - Procedures perform a specific task and don’t return a value

- Python uses functions to do both
- Modules tend to be groups of functions that perform several tasks

- Functions make long sequences of code manageable for efficiency, maintenance and re-use

def() ‘function declaration’ jargon

def funct_name ( ) :
    indented code executes here

def funct_name(): is the ‘function header’
- begin with ‘def’
- then user-defined name #any name
- then parentheses () #empty for now
- then a colon :

Indent is automatic – all indented code executes

- Creates a ‘user-defined’ function
  - It executes its indented code
- It is ‘called’ by a ‘caller’ to execute
- When all indented code has executed
  - control returns to the statement after the caller

non-modular vs modular

Non-modularised

```
#no code above the menu code - yet


print ('Main Menu')
print ('A: Cat Age')
print ('B: Dog Age')
print ('C: Goldfish Age')
print ('X: Exit')
option = input ('Enter option: ')
option = option.upper()

if option == 'A':
    print ('Under development')
elif option =='B':
    print ('Under development')
elif option =='C':
    print ('Under development')
else:
    print ('Exit program')
```

Lot of repeated code all doing the same thing

Modularised (not finished yet)

```
def stub ():                          #python function
    print ('Under development')


print ('Main Menu')                   #not a function - yet
print ('A: Cat Age')
print ('B: Dog Age')
print ('C: Goldfish Age')
print ('X: Exit')
option = input ('Enter option (A, B or X): ')
option = option.upper()

if option == 'A':
    stub()
elif option =='B':
    stub()
elif option =='C':
    stub()
else:
    print ('Exit program')
```

Code one function def stub() and 'call' it as many times as needed:
're-usable code'

Modularised – flow of control

```
def stub ():  #function to be called
    print ('Under development')


print ('Main Menu')
print ('A: Cat Age')
print ('B: Dog Age')
print ('C: Goldfish Age')
print ('X: Exit')

option = input ('Enter option (A, B or X): ')
option = option.upper()

if option == 'A':
    stub()   #caller
elif option =='B'
    stub()   #caller
elif option =='C':
    stub()   #caller
else:
    print ('Exit program')
```

We have defined a function called 'stub' #can be any name;
Functions defined at top of code after any import statements;
Can have many functions.

Flow of control:
1. 'call' the function via a 'caller' – the line of code that uses its name with parentheses (blank parentheses for now)

2. Called function begins execution of its indented code
Here we output a string Under development

3. When function execution is complete:
    3.1 restart control flow at next statement after the caller e.g.
        i/p = 'A' - 'call' function 'stub()'
        When function stub() is finished
            execute next line after the 'caller' i.e.
                execute 'elif option==B:'

Program starts with main() and def main():

```
#import( ) if being used goes before function definitions – ensures library functions are available to all following functions
def cat_months ( ):
        indented code for function here
def cat_years ( ):
        indented code for function here
def main ( ) :  #must have this  - always called 'def main'
        print ("Main Menu")
        print ("A: Cat in Months")
        print ("B: Cat in Years")
        print ("X: Exit")
        option = input ("Enter an option: ")
main()   #must have this – always main()  -start of program
```

ALWAYS
def main(): below other functions and before main()

ALWAYS
main() as the last statement after def main():
Starts the program with a call to def main() above it
- Unindented
- Parentheses ()
- No colon

Calling a function

```
def howToDoIf():                                    #step-1
    print ("Option-1: if")
    x = int(input("Enter an integer: "))
    if (x < 0):
        print ("Number is negative")

def howToDoIfElse():                                #step-1
    print ("Option-2: if-else")
    x = int(input("Enter an integer: "))
    if (x < 0):
        print ("Number is negative")
    else:
        print ("Number is not negative")
def main():                                         #step-3
    print ("Main Menu")
    print ("1: If")
    print ("2: If-else")
    option = input("Enter an Option: ")
    if option == "1":                               #step-4
        howToDoIf()
    elif option == "2":                             #step-5
        howToDoIfElse()
    else:
        print ("Exit")
main()                                               #step-2
```

step-1:
Starting at line-01, Python 'looks' for an unindented main() and ignores anything else

step-2: main() with no def or colon is found
main() calls def main():

step-3: def main(): has been called by main()
Execute indented code – output a menu

step-4: if i/p='1' call howToDoIf():
Function executes and finishes its task

step-5: Execution returns to line after the 'caller' elif option=="2":

Modular program Step-1
define def main(): and call it with main()

```
def main():
        print("Main Menu”)
        print ("1: Party Drinks")
        print ("2: Find the Cards")
        print ("X: Exit")
        print()
        option = input ("Select an option: 1, 2 or X ")
        option = option.upper()
main()
```

Good practice:
main() is the last line of code;

Above main() is def main():

Above def main() are all the other def()’s

Above all the other def()’s is import (if it is being used)

Modular program Step-2
Add a while loop

```
def main ( ):
    option = ‘ ‘
    while option !="X“:
        print("Main Menu”)
        print("1: Party Drinks")
        print("2: Find the Cards")
        print("X: Exit")
        print()
        option = input ("Select an option: 1,2 or X ")
        option = option.upper()


#start
main ( )
```

Make a menu a ‘single-entry-single-exit’ system:

User can only enter the program via a main menu;

When their chosen option has executed, show menu again;

User can only exit the program via ‘X’ on the main menu
i.e. don’t allow user to exit your program anywhere else

Modular program Step-3
add selection and validate input

```
def main():
    option = ""

    while option != "X":
        print ("Main Menu")
        print ("---------")
        print ("1. Party Drinks")
        print ("2. Find the cards")
        print ("X: Exit")
        print ()
        option = input ("Select option 1,2 or X: ")
        option = option.upper()

        while option not in ("1", "2", "X"):
            print ("Invalid option")
            option = input ("Select 1, 2 or X: ")
            option = option.upper()

        if option == "1":
            partyDrinks()
        elif option == "2":
            findTheCards()
#START
main( )
```

if needs better structure due to slide space

Modular program Step-4
Add the functions and test them

```
def partyDrinks():
    print (“Stub: Party Drinks selected")

def findTheCards():
    print (“Stub: Find the Cards Selected")

def main ( ):
    option = "“
    while option !="X“:
        print("Main Menu”)
        print("1: Party Drinks")
        print("2: Find the Cards")
        print("X: Exit")
        print()
        option = input ("Select an option: 1,2 or X ")
        option = option.upper()
         if option == "1":
            partyDrinks()
         if option == "2":
            findTheCards()
main()
```

Use stubs at this stage to check the call is working ok

- A called function must be defined above its caller before it can be called, i.e. we can’t call a non-existent function
- A called function will execute all its indented code
- When function ends, execution returns to the line after the caller

if needs better structure due to slide space

Modular program Step-5
More loops needed anywhere?

```
def partyDrinks():
    runAgain = "Y“
    while runAgain == "Y":
        print ("Option-1 Party drinks selected")
        runAgain = input ("Run this again (Y/N): ")
        runAgain = runAgain.upper()

def findTheCards():
    print ("Find the Cards Selected")

def main ( ):
    option = "“
    while option !="X“:
    print("Main Menu”)
    print("1: Party Drinks")
    print("2: Find the Cards")
    print("X: Exit")
    option = input ("Select an option: 1,2 or X ")
  option = option.upper()
     if option == "1":
         partyDrinks()
     if option == "2":
         findTheCards()
main ( )
```

Make function partyDrinks (): iterate at least once

Ask user again (in the loop) if they want to run it again
When they press ‘N’, execution of the function is finished and control returns to the main menu

At the moment the function is behaving as a ‘stub’ - it is easier to test the iteration while there is not much code to confuse things

Function findTheCards(): runs once only when it is called

if needs better structure due to slide space

Starting a program

Start with main() for single module

```
def optionA():
        print ('This is option-A')
def optionB():
        print ('This is option-B')
def main():
        print ('A: optionA')
        print ('B: optionB')
        print ('X: exit')
        option = input('Enter an option: ')
        if option == 'A':
                optionA()
        elif option == 'B':
                optionB ()
        else: print ('Exit')
main()
```

Start with if __name__ for integrated modules

```
def optionA():
        print ('This is option-A')
def optionB():
        print ('This is option-B')
def main():
        print ('A: optionA')
        print ('B: optionB')
        print ('X: exit')
        option = input('Enter an option: ')
        if option == 'A':
                optionA()
        elif option == 'B':
                optionB ()
        else: print ('Exit')

if __name__ == '__main__':
    main()
```

if __name__ == '__main__':

Information only

Python 3 upward

- Python recognises a .py file as a 'module'
  - Can be 'stand alone' or can be imported from another module
- Execution begins at the main() function
  - A special var called __name__ is defined by Python
    - When a file executes, it runs as the main program and its name is allocated to __name__
  - The main program is the one that imports other modules
    - It is defined as __main__
      - It checks if a function is running from the main program or from an imported one

__name__ and __main__ use two underscores, one after the other with no spaces.
The double underscore is referred to as 'dunder' (**d**ouble **under**score)

There are many 'dunder methods' ('magic methods' as they have no programmer intervention)